Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistemas Operativos 1

# Práctica No. 3 Exclusión Mutua

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Marzo de 2026.

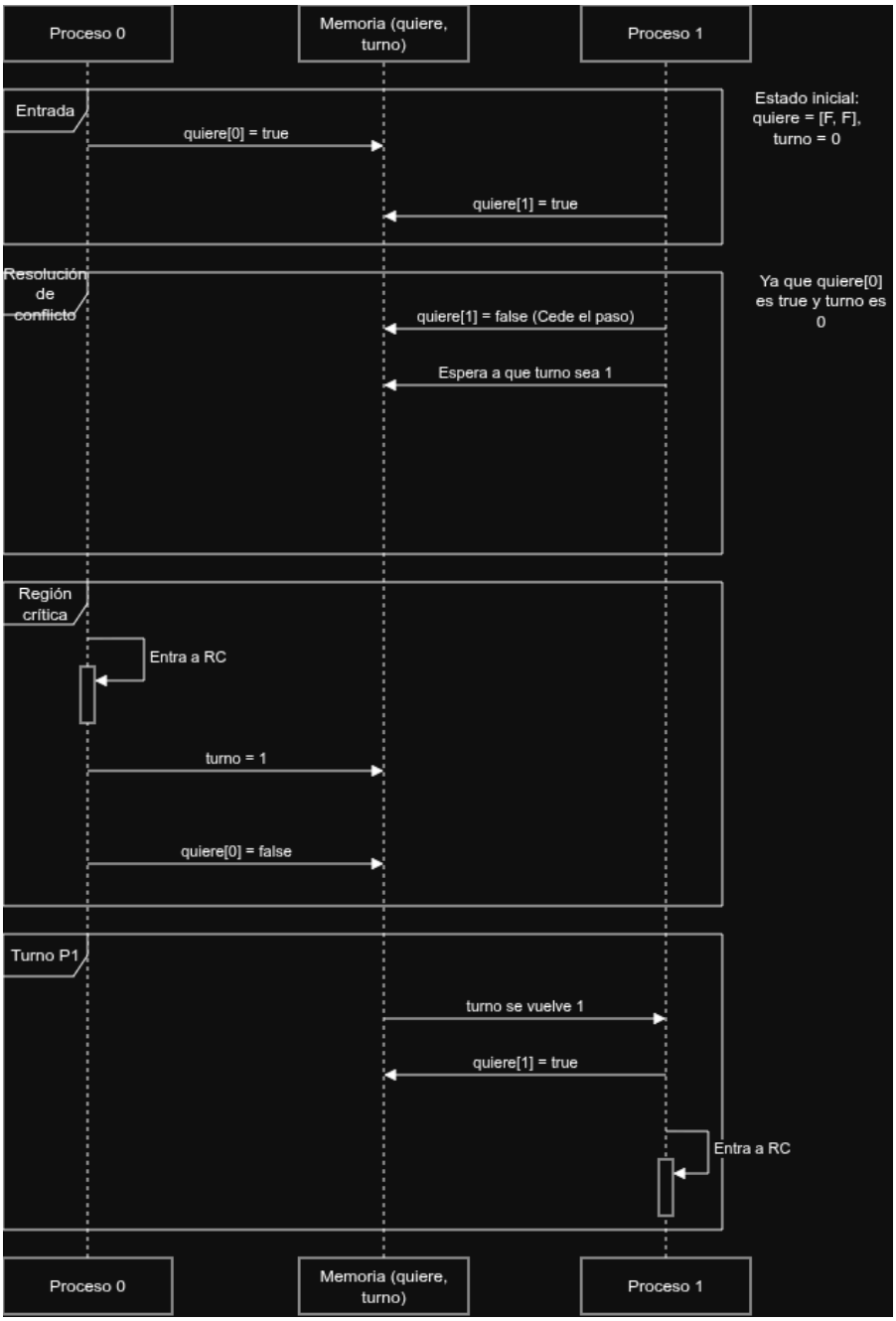
# Práctica Exclusión Mutua

## Ejercicio 1

```
└─ ./practica3
Ejercicio 1
==============================================

--- ESCENARIO 1: Operación Normal ---
--- Intento de acceso: Sucursal 1 ---
[ACCESO] Sucursal 1 procesando pedido en bodega...
[LISTO] Sucursal 1 finalizó su solicitud.
[SISTEMA] Prioridad transferida a Sucursal 2

--- Intento de acceso: Sucursal 2 ---
[ACCESO] Sucursal 2 procesando pedido en bodega...
[LISTO] Sucursal 2 finalizó su solicitud.
[SISTEMA] Prioridad transferida a Sucursal 1

--- ESCENARIO 2: Sucursal 1 entra en mantenimiento ---
NOTA: La Sucursal 1 no operará hoy, por lo tanto no cambiará el turno.
--- Intento de acceso: Sucursal 2 ---
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
[BLOQUEADA] Sucursal 2 esperando su turno. Turno actual de: Sucursal 1
```

El sistema se detiene únicamente mediante una interrupción externa, ya que el Algoritmo 1 no garantiza la propiedad de progreso. Si un proceso falla o se detiene fuera de su región crítica, bloquea permanentemente a los demás procesos que desean entrar.

# Ejercicio 2




## Escenarios

**Escenario 1 (Camino Directo)**

- **El estado inicial de quiere[0], quiere[1] y turno que permite el acceso directo.**

  Para que el proceso 0 tenga acceso directo, el estado debe ser quiere[0] = true, quiere[1] = false. El valor es irrelevante en este primer instante porque no hay oposición.

- **¿Por qué el proceso no entra al bucle de espera activa y puede proceder inmediatamente a la región crítica?**

  El proceso ejecuta la instrucción while(quiere[1]). Como la bandera del proceso 1 es false, la condición del bucle es false desde el inicio. Al no cumplirse la condición, el programa salta el cuerpo del while y entra directamente a la Región Crítica (RC).

- **¿Por qué este escenario representa el caso de menor costo computacional y qué implicación tiene en términos de rendimiento respecto a los otros escenarios?**

  Es el caso de menor costo porque el procesador solo realiza una comparación lógica y un salto. No se desperdician ciclos de CPU en ciclos infinitos. En términos de rendimiento, este es el escenario ideal ya que la latencia de entrada a región crítica es mínima.

**Escenario 2 (Un proceso bloqueado)**

- **El estado de las variables quiere[0], quiere[1] y turno en cada momento relevante.**
  - Proceso cero en región crítica: quiere[0] = true, quiere[1] = false, turno = 0.
  - Proceso 1 intenta entrar: cambia quiere[1] = true.
  - Bloqueo de proceso 1: al ver quiere[0] == true y turno == 0, proceso 1 cambia quiere[1] = false y espera.
- **El mecanismo por el cual el proceso bloqueado realiza espera activa y en qué condición abandona dicha espera.**

  Proceso 1 entra en un bucle while (turno != 1). El proceso se queda atrapado ejecutando esa línea repetidamente, consumiendo CPU solo para verificar el valor de la variable. Abandona esta espera únicamente cuando detecta que “turno” ha cambiado a 1.

- **Cómo el proceso que sale de la región crítica transfiere el control al proceso en espera, detallando qué variables se modifican y en qué orden.**

  El proceso que sale de proceso 1 transfiere el control realizando dos acciones en este orden estricto:

  - turno = 1: cambia el árbitro para que el proceso en espera pueda romper su bucle interno.
  - quiere[0] = false: baja su bandera de interacción para indicar que la región crítica está libre.

**Escenario 3 (Ambos procesos compiten al mismo tiempo)**

- **La secuencia exacta de estados de quiere[0], quiere[1] y turno desde el momento del conflicto hasta la resolución.**
  - quiere[0]=true, quiere[1]=true → atmos levantan la bandera.
  - Ambos entran al while principal, suponiendo que turno=0.
  - Proceso 1 entra al if (turno != 1), cambia quiere[1]=false y entra en espera activa.
  - Proceso 0 mira que quiere[1] ahora es false, rompe su while y entra a la región crítica.
  - Al salir, proceso 0 pone turno=1 y quiere[0]=false.

    - Proceso 1 detecta turno=1, pone quiere[1]=true y entra a la región crítica.
- **¿Por qué el proceso cuyo turno no corresponde debe bajar su bandera y entrar en espera?, y qué condición de salida lo reactiva.**

  El proceso que no tiene el turno debe bajar su bandera para evitar un interbloque. Si ambos mantuvieran la bandera arriba, ninguno dejaría entrar al otro. Se reactiva cuando el turno cambia a su favor.

- **Argumente por qué ningún proceso puede quedar bloqueado indefinidamente dado el valor inicial de turno y el número de iteraciones.**

  No hay bloqueo indefinido porque el algoritmo garantiza la finitud. Al terminar su región crítica, cada proceso está obligado por código a cambiar el turno al oponente. Como el número de iteraciones es finito (3 y 5), el proceso que tiene el turno eventualmente terminará y lo cederá, permitiendo que el otro progrese.

- **Identifique si existe riesgo de condición de carrera sobre la variable contador bajo algún escenario de ejecución con el código tal como está escrito**

  Aunque la variable contador se incrementa mediante una copia local, el algoritmo de Dekker garantiza la exclusión mutua estricta. Esto significa que es físicamente imposible que ambos hilos ejecuten esas líneas al mismo tiempo, por lo tanto, el acceso al contador es siempre secuencial y seguro.

# Anexos

Link a github.